

CSE4006

DEEP LEARNING

MODULE-1

Linear Algebra

Linear Algebra-Scalars, Vectors, Matrices and Tensors-
Operations –Special kind of Matrices and Vectors-
Probability Basics-Gradient based optimization- Linear
Classifiers, Linear Machines with Hinge Loss-
Introduction to Neural Network-Activation functions-
Tensorflow , keras operations

Linear Algebra

- Linear algebra is the branch of mathematics that focuses on linear equations.
- It is often applied to the science and engineering fields, specifically machine learning.
- Linear algebra is also central to almost all areas of mathematics like geometry and functional analysis.
- Data is represented by linear equations, which are presented in the form of matrices and vectors.

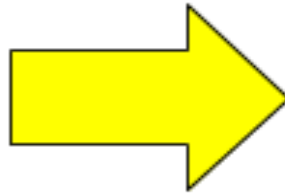
- Operations on the image, such as cropping, scaling, shearing, and so on are all described using the notation and operations of linear algebra.

One Hot Encoding

- It is common to encode categorical variables to make them easier to work and learn by some techniques.
- A popular encoding for **categorical variables** is the one hot encoding.
- A one hot encoding is where a table is created to represent the variable with one column for each category, and a row for each example in the dataset.
- A check, or one-value, is added in the column for the categorical value for a given row, and a zero-value is added to all other columns.

Example for one hot encoding

Color
Red
Red
Yellow
Green
Yellow



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

Mathematical Objects

Scalar:

24

Vector:

[2, -6, 9]

row

or
column

**2,
6,
9**

Matrix:

**2, -6, 9
4, 5, -7**

row(s) x column(s)

Scalar

- Scalars are just numbers. They have magnitude but no direction.
- Scalars are commonly used to denote quantities such as temperature, time, mass, or distance.
- Any single value from the dataset would represent a scalar.
- E.g., The number of bedrooms in the house price data would be a scalar.
- If we only used one feature as an input from house price data, then we could represent that as a scalar value

```
1 house_price_df.iloc[[0]]
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built
0	7129300520	20141013T000000	221900.0	3	1.0	1180	5650	1.0	0	0	...	7	1180	0	1955

1 rows x 21 columns

```
print(house_price_df['bedrooms'].values[0])  
3
```

- This is an example for scalar data
- Learning Rate:
 - The learning rate is a scalar value that determines the step size during the optimization process of training a deep learning model.
 - It controls how quickly or slowly the model adapts to the training data.
 - Adjusting the learning rate influences the convergence and performance of the model.
- Activation Functions:
 - Activation functions, such as the sigmoid or ReLU, take scalar inputs and apply non-linear transformations to introduce non-linearity in neural networks.
 - These scalar-valued functions play a critical role in enabling the network to learn complex patterns and relationships in the data.

- Bias Term:

- In deep learning, a bias term is often included in each layer of a neural network.
- This scalar value is added to the weighted sum of inputs and activation functions, allowing the network to learn offset or shift from the origin.
- The bias term helps the model capture more **nuanced patterns** and improves its expressive power.

- Confidence Scores:

- In classification tasks, deep learning models produce scalar-valued confidence scores or probabilities to indicate the model's confidence in its predictions.
- These scalar values reflect the model's belief in the predicted class and play a crucial role in decision-making and evaluating the model's performance.



Vectors

- A vector is a mathematical object that represents a collection of values or features.
- It is an ordered list of numbers
- Each number corresponds to a specific component or dimension of the vector.
- Vectors in deep learning are typically represented as column vectors, meaning they are written vertically.
- A vector in deep learning is denoted as $x = [x_1, x_2, x_3, \dots, x_n]^T$
- Here, x represents the vector, and $x_1, x_2, x_3, \dots, x_n$ are the individual components of the vector.
- The superscript T indicates the transpose operation, which converts a row vector into a column vector.

Example

- There seems to be more than one usable feature in our house price data.
- How would we represent multiple features?
- The total square footage of the house would be a useful piece of information to have when trying to predict a house price.
- In its most simple format, we can think of a vector as a 1-D data structure

```
# This is a numpy row vector  
print(house_price_df[["bedrooms", "sqft_lot"]].values[:1])  
[[ 3 5650]]
```

- We can also create a 1-D vector by arranging the data in a column, rather than row, format:

```
# This is a numpy column vector  
print(house_price_df[["bedrooms", "sqft_lot"]].values.T[:2, :1].shape)  
[[ 3]  
 [5650]]
```

Vector operations

- Element-wise multiplication involves multiplying each corresponding element of two vectors together.
- This operation is commonly used in various deep learning applications, such as **attention mechanisms**, **weighted computations**, and **element-wise interactions**.
- Example 1:
 - Vector A: [2, 3, 4] Vector B: [1, 2, 3]
 - Element-wise multiplication, $A \odot B = [2*1, 3*2, 4*3] = [2, 6, 12]$
- Example 2:
 - Vector X: [1, 2, 3] Vector Y: [4, 5, 6]
 - Dot product, $X \cdot Y = (1*4) + (2*5) + (3*6) = 4 + 10 + 18 = 32$

Matrices

- Matrices play a significant role in deep learning as they provide a structured way to represent and manipulate data.
- In deep learning, matrices are used to represent various components such as **input data, weights, activations,** and **gradients** in neural networks.
- They enable efficient computation and manipulation of data within neural networks, supporting various applications across **image processing, natural language processing, recommender systems,** and **generative modeling**

Examples:

Image Processing: Used to represent images in deep learning for tasks like **image classification**, object detection, and image generation.

Natural Language Processing: Matrices are employed to represent textual data in tasks such as **sentiment analysis**, **language translation**, and **text generation**.

Recommender Systems: Matrices are utilized in collaborative filtering-based recommendation systems to represent user-item interactions and calculate **similarity** or **preference scores**.

Generative Models: Matrices are used in generative models like Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) to **generate new samples** by manipulating latent space vectors.

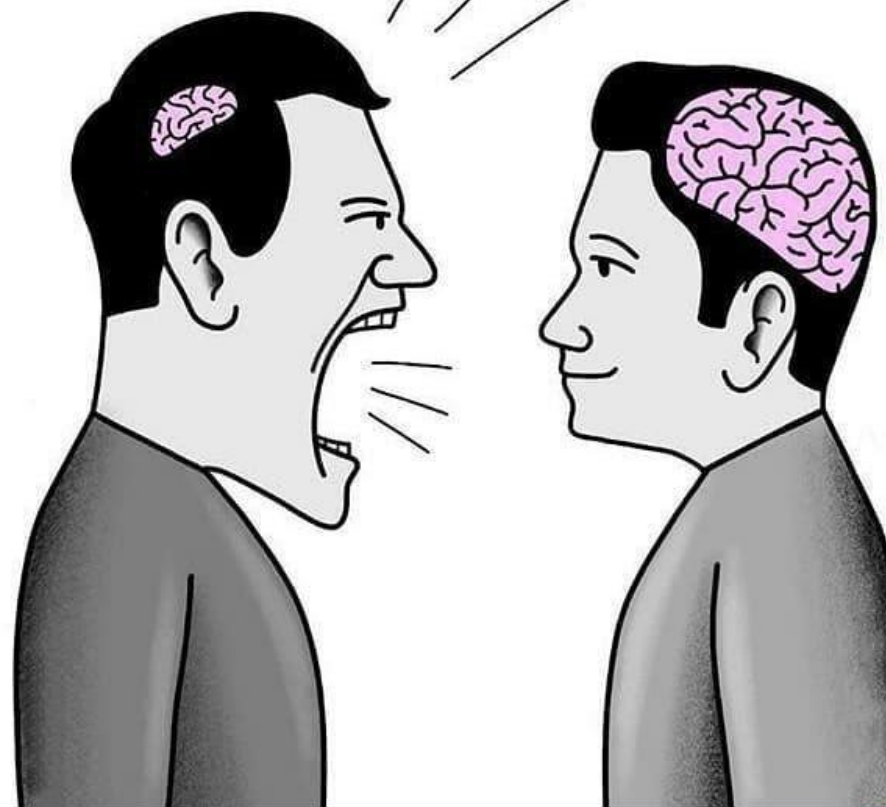
Matrix operations

- **Element-wise Operations:** Element-wise operations are performed on corresponding elements of two matrices **or** a matrix and a scalar.
- **Matrix multiplication**, and **element-wise operations** are key tools in deep learning that enable **efficient** computation, manipulation, and **transformations** of data within neural networks.



A tensor is a matrix

A tensor is an algebraic object that describes a relationship between sets of algebraic objects related to a vector space...



Tensor

- A tensor can be a **generic structure** that can be used for **storing, representing, and changing data**.
- Tensors are fundamental data structure used by all machine and deep learning algorithms.
- In the general case, an array of numbers arranged on a **regular grid** with a **variable number of axes** is known as a tensor
- A **tensor** is just a container for data, typically numerical data.
- Tensors are multi-dimensional arrays that generalize the concept of vectors and matrices.
- The matrices are called 2D tensors (in the context of tensors, a dimension is called an axis).
- In deep learning, tensors are a fundamental data structure used to represent and process data of various types and dimensions.

Common tensor representations:

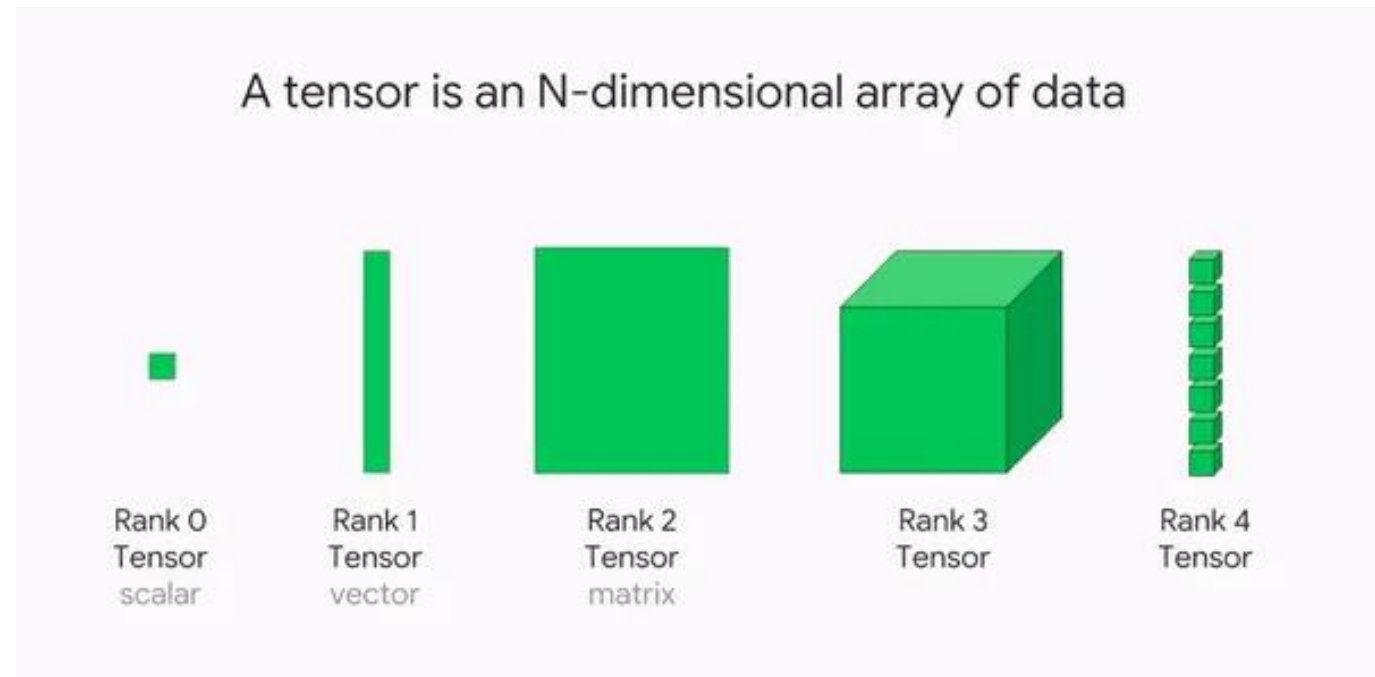
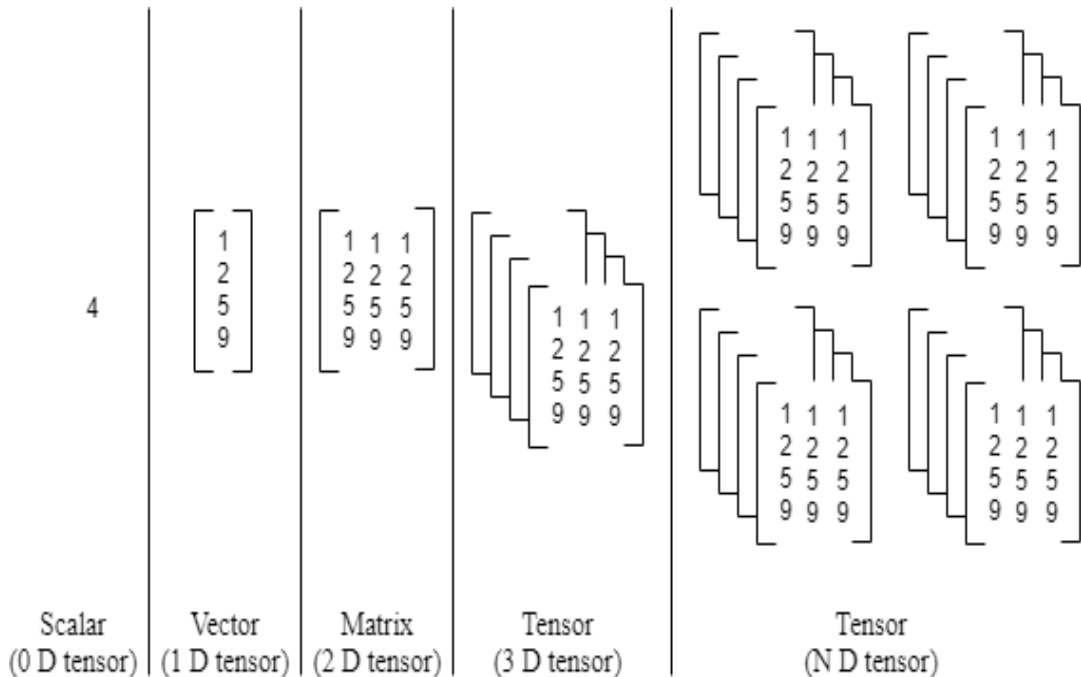
- **Vectors:** 1D — (features)
- **Sequences:** 2D — (timesteps, features)
- **Images:** 3D — (height, width, channels)
- **Videos:** 4D — (frames, height, width, channels)

Tensor

- Three **primary attributes** define a tensor:
 - Rank
 - Shape
 - Data type

Examples:

- The rank of a matrix is 2 because it has two axes.
- The rank of a vector is 1 because it has a single axis.
- The shape of a tensor refers to the number of dimensions along each axis.



Tensor

- A tensor has three indices, where the first one points to the row, the second to the column and the third one to the axis.
- For example, T232 points to the second row, the third column, and the second axis.



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	15
16	17	17	18
19	20	21	22

tensor of dimension
[6, 4, 2]

Indexes required to access an element

The relationship between each of these pairs is that they both require the same number of indexes to refer to a specific element within the data structure.

Indexes required	Computer science	Mathematics
0	number	scalar
1	array	vector
2	2d-array	matrix
n	nd-array	nd-tensor

Examples for tensors

- **Images** are commonly represented as tensors.
- A **grayscale image** can be represented as a 2D tensor, where each element represents the pixel intensity.
- **Color images** are represented as 3D tensors, with each element storing color information for each pixel (e.g., RGB channels).
- **Text** or **time-series** data, such as sentences or audio, can be represented as tensors.
- A **sentence** can be encoded as a 2D tensor, with each row representing a word and each column representing a feature.
- Time-series data can be represented as a 3D tensor, with **dimensions** representing **time steps**, **features**, and **samples**.
- The weights and biases in neural networks are represented as tensors. The dimensions of weight tensors depend on the architecture and layer sizes.
- These tensors store the learnable parameters that are updated during the training process.

E.g., vector data: 2D tensor data

- A statistical dataset of consumers, where each individual's age, height, and gender are taken into account.
- Since each individual may be represented as a vector of three values, the full dataset of 100 individuals can be stored in a 2D tensor of the shape (100, 3).
- A collection of textual information in which each article is represented by the number of times each **word occurs** in it (out of a dictionary of 2000 common words).
- A full dataset of 50 articles can be kept in a tensor of shape (50, 2000) since each article can be represented as a vector of 2000 values (one count per word in the dictionary).

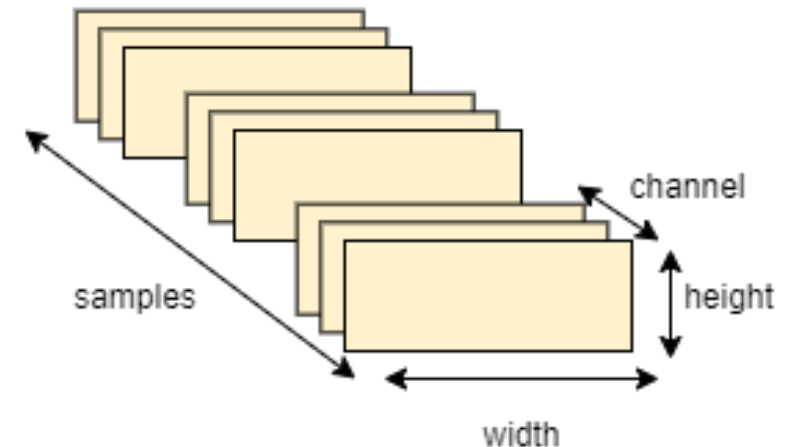
Example : 3D tensor data

- By definition, image tensors are always 3D, with a one-dimensional colour channel for grayscale images.
- Even though grayscale images (like our MNIST digits) only have a single colour channel and may therefore be stored in 2D tensors.
- Thus, a tensor of shape (32, 64, 64, 1) might be used to save a batch of 32 grayscale photos of size 64 x 64, while a tensor of shape (32, 64, 64, 3) could be used to store a batch of 32 colour images (32, 64, 64, 3).
- 5D tensor that includes x position, y position, R/G/B, person, and expression



Tensorflow and Theano

- Tensorflow takes channels as the last format.
- **Theano** uses channels-first format
- The colour-depth axis is located at the end of the list of dimensions in the **Google TensorFlow**: (samples, height, width, colour-depth).
- The batch axis is placed first, followed by the samples, colour-depth, height, and width axes by **Theano**.
- The previous instances would be transformed into (32, 1, 64, 64) and (32, 3, 64, 64). Both file types are supported by the Keras framework

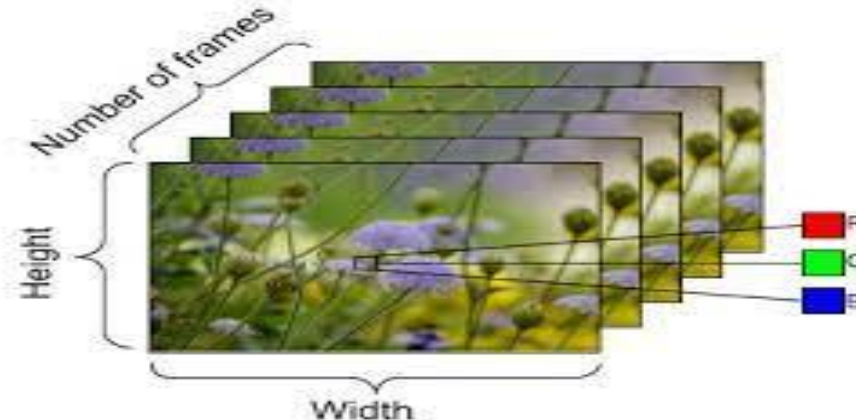


Video data

- A video could be viewed as a **set of coloured images** called **frames**.
- A batch of various movies can be saved in a **5D tensor** of shape (**samples, frames, height, width,** and **colour-depth**) since each frame can be kept in a 3D tensor (height, width, and colour-depth).
- A series of frames can also be saved in a 4D tensor (frames, height, width, and colour-depth).

Example :

- 240 frames would be present in a 60-second, 144 x 256 YouTube video clip sampled at 4 frames per second.
- Four of these video clips would be saved in a tensor shape as a batch (4, 240, 144, 256, 3).



Tensor Operations

1. Addition
2. Multiplication (including dot product and Hadamard Product)
3. Broadcasting
4. Transpose
5. Inverse

Addition

- A normal matrix addition involves element wise addition.
- Let us consider two matrices A and B, and their sum resulting in a new matrix C.

$$A_{ij} + B_{ij} = C_{ij}$$

- For addition of two matrices, the dimensions of the matrices must match. The final resulting matrix would also be of the same dimension.
- E.g.,

$$\begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 4 \\ 3 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 4 & 0 \end{pmatrix}$$

Tensor Operations

Multiplication

- Multiplication of a matrix A of dimension $m * n$ and a matrix B of dimension $n*q$ is given by:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

- For multiplication, the number of columns of the first matrix must match with the number of rows of the second matrix.
- E.g.,

$$\begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix} * \begin{pmatrix} 0 & 4 \\ 3 & 0 \end{pmatrix} = \begin{pmatrix} 3 * 0 + 2 * 3 & 3 * 4 + 2 * 0 \\ 1 * 0 + 0 * 3 & 1 * 4 + 0 * 0 \end{pmatrix} = \begin{pmatrix} 6 & 12 \\ 0 & 4 \end{pmatrix}$$

Broadcasting

- Tensor broadcasting is changing the tensors of different dimensions/shape to the compatible shape such that arithmetic operations can be performed on them.
- A special type of tensor addition operation where a **matrix** and a **vector** is added i.e., adding vector (1-D tensor) and a matrix (2-D tensor).
- **Broadcasting** is basically a programmatic approach of **adding a matrix and a vector**.
- Mathematical notation of broadcasting is: $A_{ij} + V_j = C_{ij}$, where A_{ij} is a matrix and V_j is a vector.
- The dimension of the vector should match either the number of rows **or** the number of columns of the matrix.

Example:

Let us suppose, $A = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$, $v = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Although the dimensions do not match, the vector is replicated as a part of broadcasting and the following result is produced after addition:

$$C = \begin{pmatrix} 2 & 4 & 6 \\ 4 & 6 & 8 \end{pmatrix}$$

Hadamard Product Tensor Element-wise Multiplication

- Tensor element-wise multiplication involves multiplying corresponding elements of two tensors together.
- $C_{ij} = A_{ij} \odot B_{ij}$
- This operation is useful for incorporating element-wise interactions or transformations in deep learning models.

Example:

- Tensor A: $[[1, 2], [3, 4]]$
- Tensor B: $[[2, 2], [2, 2]]$
- Resultant Tensor $C_{ij} = A_{ij} \odot B_{ij} = [[2, 4], [6, 8]]$

Dot Product Tensor Element-wise Multiplication

- A dot product of two vectors results into a scalar. The dot product between vectors $\vec{x}$ and $\vec{y}$ is defined as (in vector notation):

$$a = \vec{x} \cdot \vec{y} \quad ; \quad a = \sum_i x_i y_i$$

- where, a is scalar.

E.g.,

$$\vec{x} = [1 \ 2 \ 3 \ 4] \text{ and } \vec{y} = [2 \ 3 \ 1 \ 4]$$

$$a = \vec{x} \cdot \vec{y} = [1 \ 2 \ 3 \ 4] \cdot [2 \ 3 \ 1 \ 4]$$

$$= [1 * 2 + 2 * 3 + 3 * 1 + 4 * 4] = 27$$

In matrix representation, dot product is written as

$$a = x^T \cdot y \quad \text{or} \quad a = x \cdot y^T$$

Transpose

- The transpose of a matrix A which denoted by $B = A^T$, is given by $B_{ji} = A_{ij}$.

Example:

- Let's suppose a matrix $A = \begin{pmatrix} 4 & 5 & 3 \\ 1 & 2 & 7 \end{pmatrix}$
- The transpose $B = A^T = \begin{pmatrix} 4 & 1 \\ 5 & 2 \\ 3 & 7 \end{pmatrix}$

Tensor Concatenation

- Tensor concatenation is the process of combining tensors along a specified axis to form a larger tensor.
- This operation is commonly used to merge or combine tensors in different ways.

Example:

- Tensor A: $[[1, 2], [3, 4]]$
- Tensor B: $[[5, 6], [7, 8]]$
- Resultant Tensor C (Concatenated along axis 0):
 $[[1, 2], [3, 4], [5, 6], [7, 8]]$

Differences between Matrices and Tensors:

	Matrices	Tensors
Dimensions	Two-dimensional	Arbitrary number of dimensions
Elements	Scalar elements	Elements can be scalars, vectors, matrices, or other tensors
Application	Used in linear algebra, systems of linear equations	Used in deep learning for representing and processing complex data structures
Rank	Rank 2	Rank can be higher than 2
Shape	Represented as (m, n)	Shape specified by the size along each dimension
Examples	[[1, 2], [3, 4]]	[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]



1. Which of the following would qualify as a tensor?

- 2d array
- Vector
- Matrix
- All of these

2. How many dimensions does the tensor $z = [1, 2, 3, 4]$ have?

3. The variable y below is an example of a _____.

$$y = [[4, 2], [3, 1], [0, 8], [6, 1]]$$

Possible questions

- Linear Algebra basic questions
- Problems based on matrices, tensors and vector operations